

Informe Microservicios

Asignatura: Desarrollo Fullstack 1

Sección: 002D

Integrantes:

Marcelo Fernández

Benjamin Erpel

Joy Castillo

Introducción	2
Contexto del caso	2
Contenidos de la experiencia 2:	2
1. Persistencia de Datos y Modelado Relacional (JPA/Hibernate)	2
2. Comunicación entre Microservicios (Arquitectura Distribuida)	2
3. Seguridad Básica y Gestión Global de Excepciones	3
Modelo Arquitectura:	4
Autenticación por tokens (JWT)	4
Cómo se implementa la autenticación por tokens	4
Spring Security y Protección de Rutas	5
CORS	6
Implementación por capas	6
1. Evidencias (Usando POSTMAN)	7
2. Comunicación entre microservicios	9
Testing	9
Es fundamental ya que:	9
Pruebas unitarias AAA	10
Ejemplo de cada método implementado	10
DOCUMENTACIÓN DE APIs CON SWAGGER	17
i. ¿Qué es Swagger y en qué consiste?	17
¿Cómo se implementa en Spring Boot?	17
Evidencia de Implementación y Ejecución (3 Microservicios)	18
Microservicio 9: Facturas (service_facturas)	19
Microservicio 1: Clientes (service_cliente)	21

Introducción

El presente informe describe el diseño e implementación de una solución basada en arquitectura de microservicios para una quesería. El sistema busca optimizar la gestión de clientes, productos, inventario, pedidos y más, mediante servicios independientes desarrollados con Spring Boot.

Contexto del caso

La Pyme “Quesería sin tí Maipú” se dedica a la producción y venta de quesos artesanales. Actualmente enfrenta dificultades en su gestión debido a procesos poco automatizados.

Para mejorar la organización y eficiencia del negocio, se propone desarrollar un sistema basado en microservicios, permitiendo separar cada funcionalidad en servicios independientes. En esta etapa se implementará una serie de microservicios, conectados mediante un API Gateway.

Esta solución permitirá optimizar la gestión de la información, mejorar el control de las operaciones y facilitar el crecimiento futuro de la empresa.

Contenidos de la experiencia 2:

1. Persistencia de Datos y Modelado Relacional (JPA/Hibernate)

Este módulo se enfoca en la transmisión de datos temporales en memoria a un almacenamiento permanente en bases de datos relacionales dentro de una arquitectura distribuida.

Fundamentos de Persistencia: Diferencia entre datos temporales y persistentes; flujo completo de una transacción y mantenimiento de la coherencia e integridad del sistema.

Modelado de Entidades JPA: Configuración del entorno, mapeo de entidades principales y uso de anotaciones JPA.

Diseño Relacional: Incorporación de llaves primarias, llaves foráneas y tipos de relaciones comunes mediante código (@OneToMany, @ManyToOne, @ManyToMany).

Capa de Persistencia (Patrón CSR): Implementación de repositorios, modificación de *endpoints* existentes para operaciones CRUD directas en la base de datos y validación mediante herramientas REST.

2. Comunicación entre Microservicios (Arquitectura Distribuida)

Se centra en la integración y el intercambio de información entre servicios independientes dentro de un ecosistema modular y escalable.

Modelo Arquitectura:



Autenticación por tokens (JWT)

Consiste en la utilización de un **Servicio de Autenticación centralizado** encargado de gestionar los usuarios, sus roles y emitir tokens **JWT (JSON Web Tokens)**, que funcionan como un "pasaporte" digital.

Es un estándar abierto (RFC 7519) que sirve para transmitir información de forma segura entre dos partes como un objeto JSON. Su gran ventaja es que los tokens son **autocontenidos y están firmados digitalmente**. Esto permite que cualquier receptor (como un API Gateway u otro microservicio) verifique que los datos no han sido manipulados sin necesidad de consultar una base de datos externa cada vez.

Un token generado por el sistema consta de tres partes separadas por puntos:

- **Header:** Indica que es un JWT y define el algoritmo de firma
- **Payload:** Contiene la información útil del usuario, sus permisos o roles, y las fechas de creación y expiración.
- **Signature:** El sello de seguridad generado con una clave secreta (jwt.secret), asegurando que si alguien altera el Payload, el token se vuelva inválido

Cómo se implementa la autenticación por tokens

- **Base de Datos:** Se crea una base de datos independiente (db_seguridad) que almacena las tablas de usuarios, roles y usuario_rols.
- **Registro de Usuarios:** Cuando un usuario se registra a través del endpoint /auth/registro, el servicio toma la contraseña en texto plano, la encripta usando BCryptPasswordEncoder (transformándola en un hash indescifrable) y la guarda en la base de datos.
- **Inicio de Sesión:** El usuario envía un JSON con su nombre de usuario y contraseña mediante la clase DTO AuthRequest al endpoint /auth/login. El sistema valida la contraseña usando la función .matches(). Si es correcta, transforma los roles del usuario en una lista de textos (List<String>) y llama al JwtService para construir y firmar digitalmente el token con una validez específica.
- **Uso del Token (Acceso):** El cliente recibe el string del token y, para realizar cualquier otra petición protegida (como interactuar con el servicio de pacientes), debe adjuntar este pase en la cabecera HTTP bajo el formato: Authorization: Bearer <TOKEN>

Spring Security y Protección de Rutas

En el microservicio de autenticación, Spring Security se configura mediante la clase SecurityConfig utilizando un componente SecurityFilterChain. Su función es actuar como la aduana que define las reglas de acceso a las URL paso a paso:

- **Rutas públicas (.permitAll()):** Se utiliza requestMatchers("/auth/").permitAll() para abrir por completo las rutas de registro e inicio de sesión. Esto permite que cualquier usuario no autenticado acceda, ya que es el paso previo obligatorio para obtener su token.
- **Rutas protegidas (.authenticated()):** Con la regla .anyRequest().authenticated(), cualquier otra ruta del microservicio exigirá obligatoriamente que el usuario esté autenticado; de lo contrario, la petición será rechazada.

- En la arquitectura global, la protección de rutas se traslada principalmente al **API Gateway**, el cual funciona como un "Guardia". Mediante un filtro global (AuthenticationFilter), el Gateway intercepta las peticiones dirigidas a rutas protegidas, extrae el token de la cabecera y valida su firma y expiración. Si el token expira o es falso, bloquea el paso enviando un error

CORS

Cross-Origin Resource Sharing constituye el mecanismo mediante el cual el servidor flexibiliza las restricciones de la Política de Mismo Origen de manera segura. A través de cabeceras HTTP específicas, el servidor comunica explícitamente al navegador que autoriza las solicitudes provenientes de un origen externo determinado (como una aplicación frontend desplegada en un puerto específico), permitiéndole así el acceso y la lectura de sus recursos

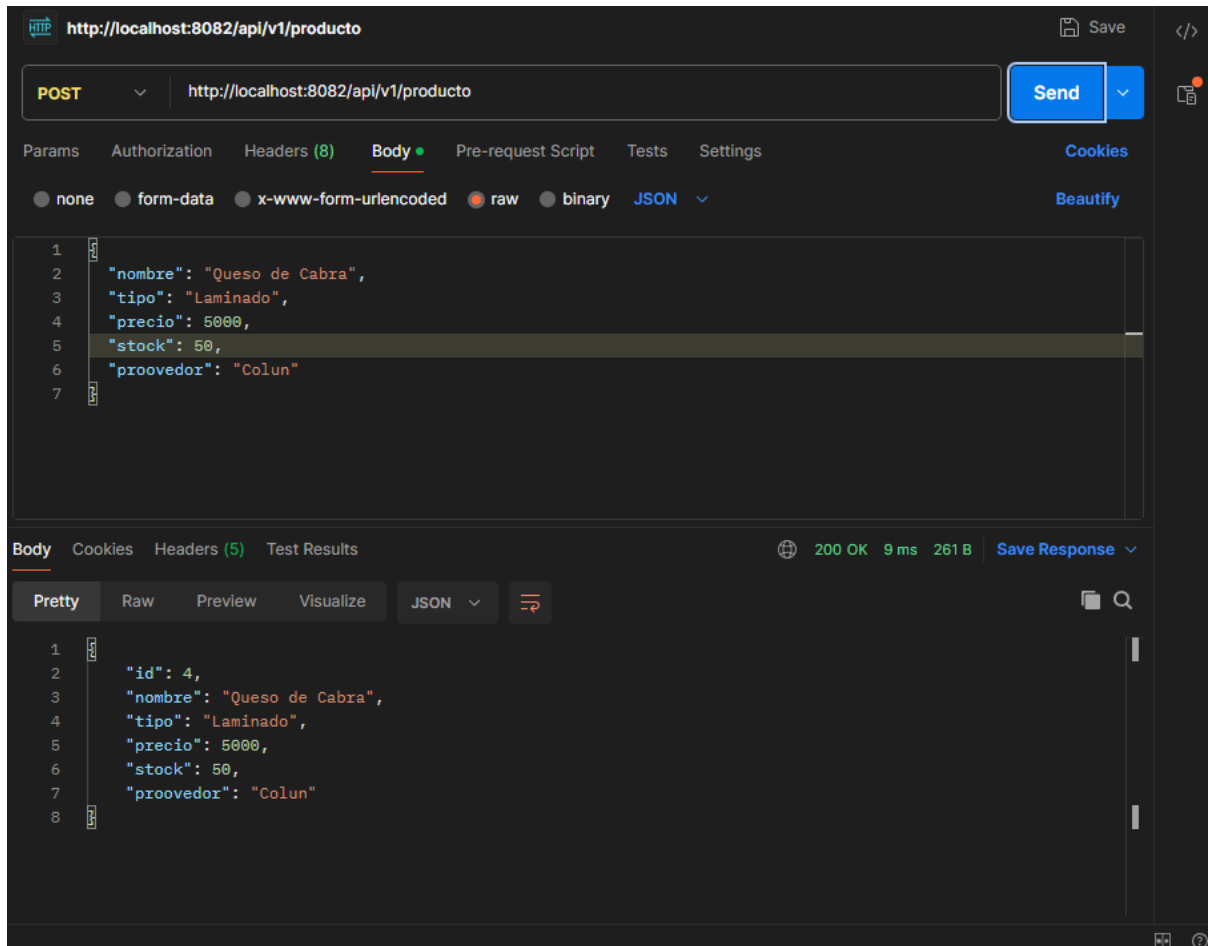
Implementación por capas

La arquitectura del microservicio se divide en responsabilidades para mantener el código ordenado y escalable, usaremos como ejemplo el microservicio de Productos para explicar las 4 capas principales y mostrar las evidencias.

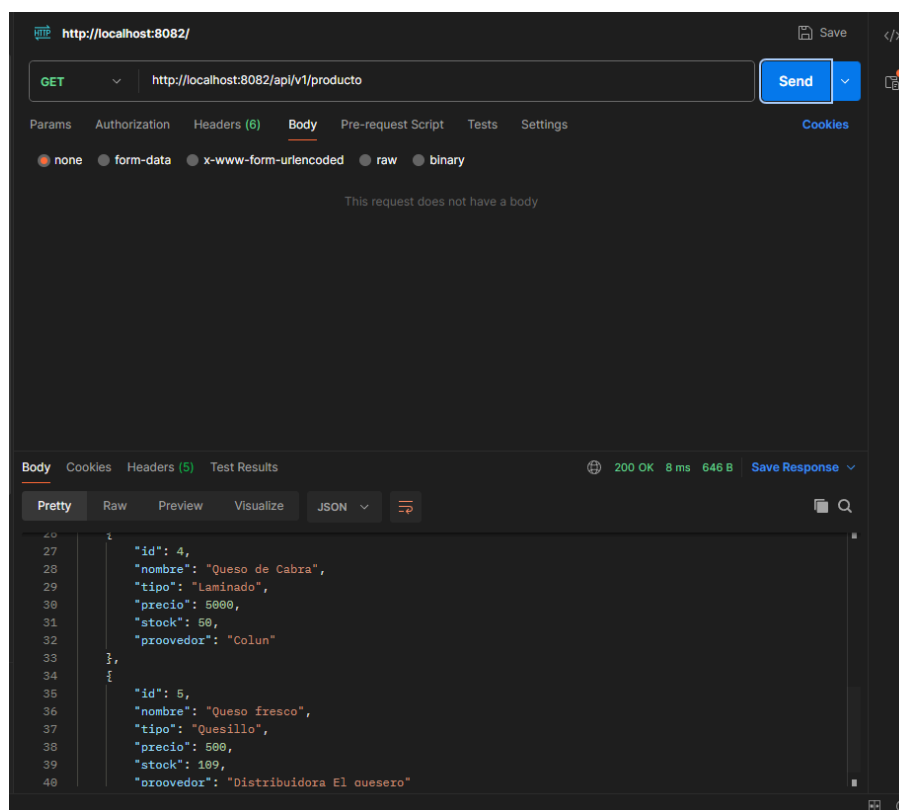
- Capa de entidad (Modelo): Representa la tabla en la base de datos (ProductosModelo)
- Capa de Repositorio: La interfaz hereda de JpaRepository y es la encargada de comunicarse con la base de datos
- Capa de Servicio: Es la capa que tiene la lógica del negocio, le comunica al repositorio que guardar/buscar
- Capa del Controlador: Es la puerta de entrada que recibe las peticiones(GET,POST,etc.) y se comunica con el servicio

1. Evidencias (Usando POSTMAN)

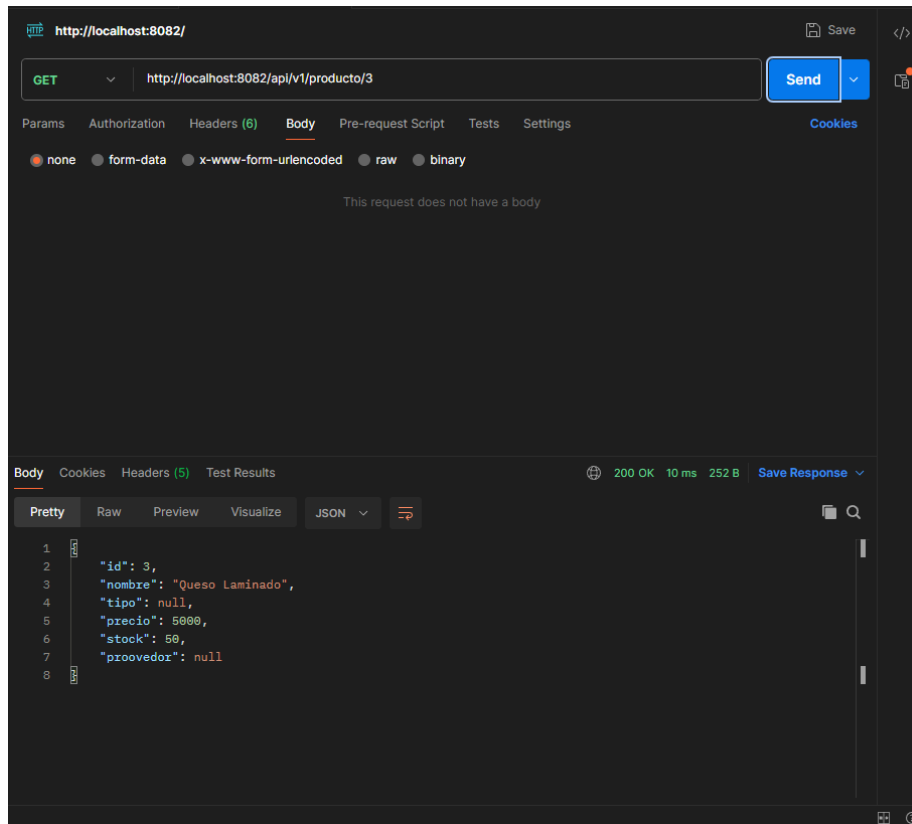
- Crear producto (POST)



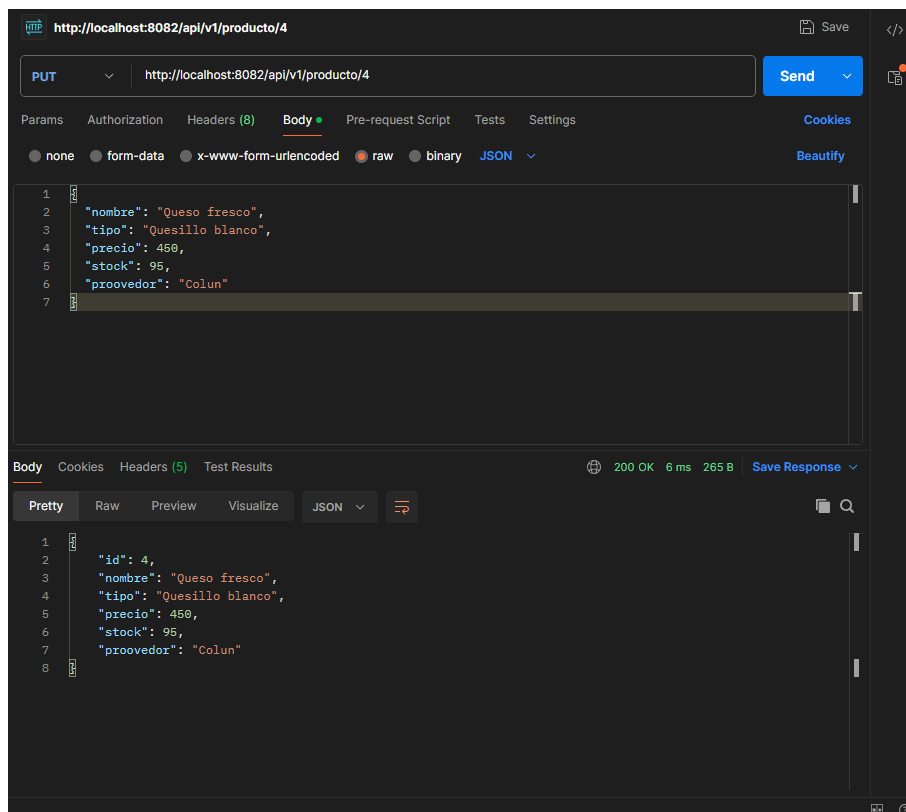
- Listar productos (GET)



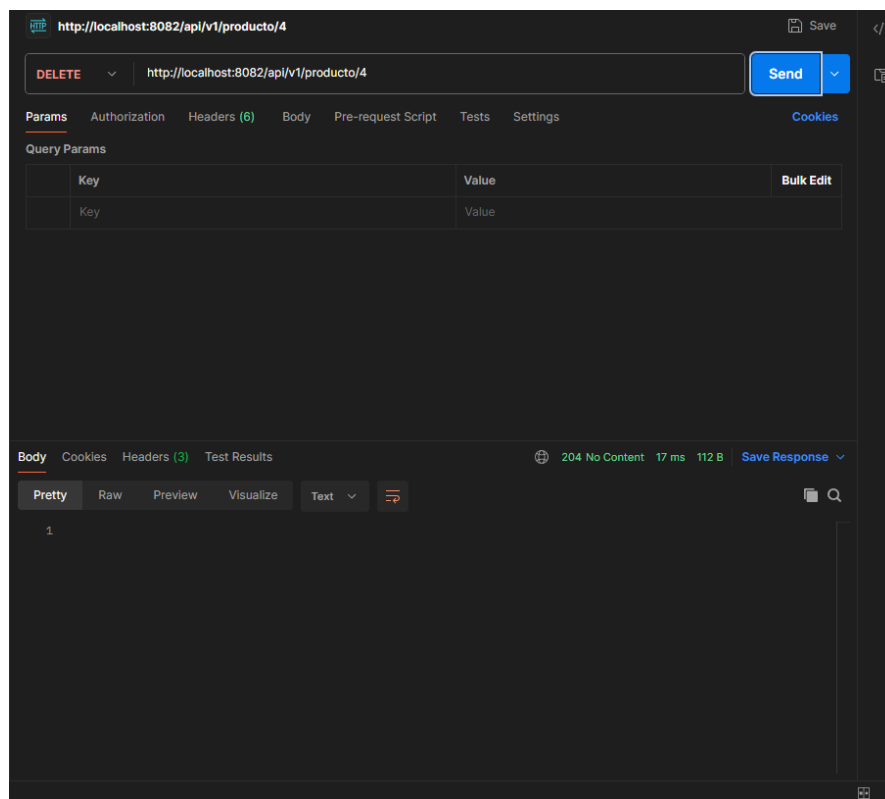
- Buscar por id (GET)



- Actualizar un producto (PUT)



- Eliminar un producto (DELETE)



2. Comunicación entre microservicios

En el contexto del negocio de la quesería, los microservicios se comunican entre sí mediante peticiones HTTP.

Por ejemplo, cuando se crea un pedido (usando el `service_pedido`), éste se comunica con el microservicio de productos (`service_productos`) para verificar si hay stock, usando las herramientas que están dentro del ecosistema de Spring.

swagger

Testing

- Consiste en ejecutar partes del código bajo condiciones controladas para verificar que hacen lo que se supone que deban hacer.

Es fundamental ya que:

- Previene errores en la producción
- Facilita la refactorización (Ejemplo: Si cambias el precio de un queso, las pruebas te informan de manera inmediata si rompió algo que antes funcionaba)
- Sirve para documentar cómo se supone que debe usarse un método

Pruebas unitarias AAA

Consiste en aislar un método de la capa de servicio y probarla sin depender de datos reales ni de otros microservicios, se estructuran usando el patrón AAA

- Arrange: Se preparan los datos de prueba, se instancian los objetos y se configuran las simulaciones
- Act: Se ejecuta el método real al que queremos testear llamando directamente
- Assert: Se comprueba que el resultado obtenido sea igual al esperado

Ejemplo de cada método implementado

```

1 package com.queseria.service_productos;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.junit.jupiter.api.Assertions.assertNotNull;
5 import org.junit.jupiter.api.Test;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.boot.test.context.SpringBootTest;
8 import org.springframework.test.annotation.Rollback;
9
10 import com.queseria.service_productos.modelo.ProductosModelo;
11 import com.queseria.service_productos.service.ProductosService;
12
13 import jakarta.transaction.Transactional;
14
15 @SpringBootTest
16 public class ServiceProductosTest {
17
18     @Autowired
19     private ProductosService service;
20
21     @Test
22     @Transactional
23     @Rollback(false)
24     public void testGuardarProducto() {
25
26         // 1. ARRANGE
27         ProductosModelo quesoPrueba = new ProductosModelo();
28         quesoPrueba.setNombre(nombre: "Queso prueba AAA");
29         quesoPrueba.setTipo(tipo: "Queso creado desde la prueba unitaria");
30         quesoPrueba.setPrecio(precio: 2500);
31         quesoPrueba.setStock(stock: 10);
32         quesoPrueba.setProveedor(proveedor: "Colun");
33
34         // 2. ACT
35         ProductosModelo resultado = service.guardarQueso(quesoPrueba);
36
37         // 3. ASSERT
38         assertNotNull(resultado);
39         assertNotNull(resultado.getId(), "El ID no debería ser nulo si se guardó en la base");
40         assertEquals("Queso prueba AAA", resultado.getNombre());
41     }
42 }

```

1. Test unitario Service Productos

```

1  package com.queseria.service_productos;
2
3  import com.queseria.service_productos.modelo.ProductosModelo;
4  import com.queseria.service_productos.repository.ProductosRepository;
5
6  import org.junit.jupiter.api.DisplayName;
7  import org.junit.jupiter.api.Test;
8  import org.junit.jupiter.api.extension.ExtendWith;
9  import org.mockito.InjectMocks;
10 import org.mockito.Mock;
11 import org.mockito.junit.jupiter.MockitoExtension;
12
13 import static org.junit.jupiter.api.Assertions.assertEquals;
14 import static org.junit.jupiter.api.Assertions.assertNotNull;
15 import static org.mockito.ArgumentMatchers.any;
16 import static org.mockito.Mockito.when;
17
18 import com.queseria.service_productos.service.ProductosService;
19
20 @ExtendWith(MockitoExtension.class)
21 class ProductosServiceTest {
22
23     @Mock
24     private ProductosRepository productosRepository;
25
26     @InjectMocks
27     private ProductosService productosService;
28
29     @Test
30     @DisplayName("Debería guardar el producto de la quesería correctamente")
31     void guardarProductoTest() {
32         // 1. Preparación (escenario)
33         ProductosModelo producto = new ProductosModelo();
34         producto.setNombre(nombre: "Queso de Oveja");
35         producto.setPrecio(precio: 8500);
36
37         when(productosRepository.save(any(ProductosModelo.class))).thenReturn(producto);
38
39         // 2. Ejecución
40         ProductosModelo resultado = productosService.guardarQueso(producto); // Ajusta si tu método se llama distinto
41
42         // 3. Verificación
43         assertNotNull(resultado);
44         assertEquals("Queso de Oveja", resultado.getNombre());
45         assertEquals(8500, resultado.getPrecio());
46     }
47 }

```

2. Test Unitario Service Cliente:

```

service_cliente > src > test > java > com > queseria > service_cliente > service > ClienteServiceTest.java > ClienteServiceTest > guardarClienteTest()
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.mockito.InjectMocks;
14 import org.mockito.Mock;
15 import org.mockito.junit.jupiter.MockitoExtension;
16
17 import com.queseria.service_cliente.modelo.Cliente;
18 import com.queseria.service_cliente.repository.ClienteRepository;
19
20 @ExtendWith(MockitoExtension.class)
21 class ClienteServiceTest {
22
23     @Mock
24     private ClienteRepository clienteRepository; // Simulamos el repositorio
25     @InjectMocks
26     private ClienteService clienteService; // Inyectamos el mock en el servicio real
27     @Test
28     @DisplayName("Debería guardar un Cliente correctamente")
29     void guardarClienteTest() {
30         // 1. Preparación (escenario)
31         Cliente cliente = new Cliente();
32         cliente.setNombre(nombre: "Jose Luis");
33         cliente.setTelefono(telefono: "+56965744315");
34         when(clienteRepository.save(any(Cliente.class))).thenReturn(invocation -> {
35             Cliente cli = invocation.getArgument(0);
36             cli.setId(id: 1L);
37             return cli;
38         });
39
40         Cliente resultado = clienteService.guardar(cliente);
41         // 3. Verificación (Then)
42         assertNotNull(resultado);
43         assertEquals(1L, resultado.getId());
44         assertEquals("Jose Luis", resultado.getNombre());
45         verify(clienteRepository, times(1)).save(cliente);
46     }
47 }

```

3. Test Unitario Service Detalle:

```

J DetalleServiceTest.java 1 X
src > test > java > com > queseria > service_detalle > service > J DetalleServiceTest.java > ...
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.mockito.InjectMocks;
14 import org.mockito.Mock;
15 import org.mockito.junit.jupiter.MockitoExtension;
16
17 import com.queseria.service_detalle.modelo.Detalle;
18 import com.queseria.service_detalle.repository.DetalleRepository;
19
20 @ExtendWith(MockitoExtension.class)
21 class DetalleServiceTest {
22
23     @Mock
24     private DetalleRepository detalleRepository; // Simulamos el repositorio
25     @InjectMocks
26     private DetalleService detalleService; // Inyectamos el mock en el servicio real
27     @Test
28     @DisplayName("Debería guardar el detalle del pedido correctamente")
29     void guardarDetalleTest() {
30         // 1. Preparación (escenario)
31         Detalle detalle = new Detalle();
32         detalle.setPrecioUnitario(precioUnitario: 12000.0);
33         detalle.setCantidad(cantidad: 2);
34         when(detalleRepository.save(any(type: Detalle.class))).thenAnswer(invocation -> {
35             Detalle deta = invocation.getArgument(index: 0);
36             deta.setId(id: 1L);
37             return deta;
38         });
39
40         Detalle resultado = detalleService.guardar(detalle);
41         // 3. Verificación (Then)
42         assertNotNull(resultado);
43         assertEquals(expected: 1L, resultado.getId());
44         assertEquals(expected: 2, resultado.getCantidad());
45         verify(detalleRepository, times(wantedNumberOfInvocations: 1)).save(detalle);
46     }
47 }

```

4. Test Unitario Service Notificaciones:

```

1  package com.queseria.service_notificaciones;
2
3  import com.queseria.service_notificaciones.model.Notificacion;
4  import com.queseria.service_notificaciones.repository.NotificacionRepository;
5  import com.queseria.service_notificaciones.service.NotificacionService;
6
7  import org.junit.jupiter.api.DisplayName;
8  import org.junit.jupiter.api.Test;
9  import org.junit.jupiter.api.extension.ExtendWith;
10 import org.mockito.InjectMocks;
11 import org.mockito.Mock;
12 import org.mockito.junit.jupiter.MockitoExtension;
13
14 import static org.junit.jupiter.api.Assertions.assertEquals;
15 import static org.junit.jupiter.api.Assertions.assertNotNull;
16 import static org.mockito.ArgumentMatchers.any;
17 import static org.mockito.Mockito.when;
18
19 @ExtendWith(MockitoExtension.class)
20 class ServiceNotificacionesApplicationTests {
21
22     @Mock
23     private NotificacionRepository notificacionRepository;
24
25     @InjectMocks
26     private NotificacionService notificacionService;
27
28     @Test
29     @DisplayName("Debería asignar estado PROCESANDO al crear la notificación")
30     void guardarNotificacionTest() {
31         // 1. Preparación (escenario)
32         Notificacion noti = new Notificacion();
33         noti.setPedidoId(pedidoId: 1L);
34         noti.setEstado(estado: "PROCESANDO");
35
36         when(notificacionRepository.save(any(Notificacion.class))).thenReturn(noti);
37
38         // 2. Ejecución
39         Notificacion resultado = notificacionService.crearNotificacion(pedidoId: 1L, clienteId: 5L);
40
41         // 3. Verificación
42         assertNotNull(resultado);
43         assertEquals(1L, resultado.getPedidoId());
44         assertEquals("PROCESANDO", resultado.getEstado());
45     }
46 }

```

5. Test unitario Service Inventario

```

1  package com.queseria.service_inventario;
2
3  import static org.junit.jupiter.api.Assertions.assertEquals;
4  import static org.junit.jupiter.api.Assertions.assertNotNull;
5  import org.junit.jupiter.api.DisplayName;
6  import org.junit.jupiter.api.Test;
7  import org.junit.jupiter.api.extension.ExtendWith;
8  import static org.mockito.ArgumentMatchers.any;
9  import org.mockito.InjectMocks;
10 import org.mockito.Mock;
11 import static org.mockito.Mockito.when;
12 import org.mockito.junit.jupiter.MockitoExtension;
13
14 import com.queseria.service_inventario.model.Inventario;
15 import com.queseria.service_inventario.repository.InventarioRepository;
16 import com.queseria.service_inventario.service.InventarioService;
17
18 @ExtendWith(MockitoExtension.class)
19 class ServiceInventarioApplicationTests {
20
21     @Mock
22     private InventarioRepository inventarioRepository;
23
24     @InjectMocks
25     private InventarioService inventarioService;
26
27     @Test
28     @DisplayName("Debería registrar el stock del queso correctamente")
29     void guardarInventarioTest() {
30         // 1. Preparación
31         Inventario registroStock = new Inventario();
32         registroStock.setProductoId(productoId: 1L); // ID del queso
33         registroStock.setCantidad(cantidad: 150); // Cantidad que ingresa o stock total
34
35         when(inventarioRepository.save(any(Inventario.class))).thenReturn(registroStock);
36
37         // 2. Ejecución
38         Inventario resultado = inventarioService.guardarInventario(registroStock);
39
40         // 3. Verificación
41         assertNotNull(resultado);
42         assertEquals(1L, resultado.getProductoId());
43         assertEquals(150, resultado.getCantidad());
44     }
45 }

```

6. Test unitario Service pagos

```
> test > java > com > queseria > service_pagos > J PagoServiceTest.java > PagoServiceTest > verificarPrecioDinamicoConDescuento()
package com.queseria.service_pagos;

import com.queseria.service_pagos.modelo.Pago;
import com.queseria.service_pagos.repository.PagoRepository;
import com.queseria.service_pagos.service.PagoService;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.any;

@ExtendWith(MockitoExtension.class)
public class PagoServiceTest {

    @Mock
    private PagoRepository pRepository;

    @InjectMocks
    private PagoService pService;

    @Test
    public void verificarPrecioDinamicoConDescuento() {
        // 1. ARRANGE (Preparar los datos de prueba)
        Long idPedido = 1L;
        Double base = 20000.0;
        Integer cant = 12; // Al ser 10 o más, aplica el 10% de descuento automático
        String met = "EFECTIVO";

        Pago pagoSimulado = new Pago(id: 1L, idPedido, base, descuentoAplicado: 2000.0, montoFinal: 18000.0, met);
        Mockito.when(pRepository.save(any(type: Pago.class))).thenReturn(pagoSimulado);

        // 2. ACT (Ejecutar el método real)
        Pago resultado = pService.registrarPago(idPedido, base, cant, met);

        // 3. ASSERT (Verificar que las matemáticas de la regla de negocio den bien)
        assertNotNull(resultado);
        assertEquals(expected: 18000.0, resultado.getMontoFinal(), message: "El monto final debería tener el 10% de descuento");
        assertEquals(expected: 2000.0, resultado.getDescuentoAplicado(), message: "El descuento debería ser de 2000");
    }
}
```

7. Test unitario Service factura

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;

public class FacturaServiceTest {

    private FacturaRepository facturaRepository;
    private FacturaService facturaService;

    @BeforeEach
    void setUp() {
        facturaRepository = Mockito.mock(classToMock: FacturaRepository.class);
        facturaService = new FacturaService(facturaRepository);
    }

    // TEST 1: Valida la lógica de negocio del cálculo del IVA (10%)
    @Test
    void testGuardarFacturaCalculaIvaCorrectamente() {
        Factura facturaInput = new Factura();
        facturaInput.setPedidoId(pedidoId: 10L);
        facturaInput.setPagoId(pagoId: 5L);
        facturaInput.setClienteRut(clienteRut: "12345678-9");
        facturaInput.setTotalNeto(totalNeto: 10000.0);

        when(facturaRepository.save(any(type: Factura.class))).thenAnswer(invocation -> invocation.getArgument(index: 0));

        Factura facturaResultado = facturaService.guardarFactura(facturaInput);

        assertEquals(expected: 1900.0, facturaResultado.getIva());
        assertEquals(expected: 11900.0, facturaResultado.getTotalFinal());
    }

    // TEST 2: Valida el listado correcto de todas las facturas
    @Test
    void testListarTodasLasFacturas() {
        Factura f1 = new Factura();
        Factura f2 = new Factura();
        List<Factura> listaSimulada = Arrays.asList(f1, f2);

        // Simulamos que el repositorio devuelve 2 facturas
        when(facturaRepository.findAll()).thenReturn(listaSimulada);

        List<Factura> resultado = facturaService.listarTodas();

        assertNotNull(resultado);
        assertEquals(expected: 2, resultado.size()); // Verifica que traiga los 2 elementos
    }
}
```


DOCUMENTACIÓN DE APIs CON SWAGGER

i. ¿Qué es Swagger y en qué consiste?

Swagger (actualmente bajo la especificación **OpenAPI**) es un conjunto de herramientas de código abierto diseñado para el diseño, construcción, documentación y consumo de servicios web RESTful.

Consiste en un sistema de anotaciones que se integra directamente en el código fuente del backend. Al compilar y levantar la aplicación, genera automáticamente una interfaz web interactiva (**Swagger UI**). En esta página, cualquier desarrollador o evaluador puede visualizar:

- Todos los endpoints disponibles (**GET**, **POST**, **PUT**, **DELETE**).
- Las rutas de acceso y los puertos de cada microservicio.
- La estructura exacta de los datos que se deben enviar y recibir (JSON).
- Un botón interactivo (*Try it out*) para probar las peticiones en tiempo real sin usar programas externos como Postman.

¿Cómo se implementa en Spring Boot?

Para implementar Swagger/OpenAPI en un ecosistema de microservicios con Spring Boot, se siguen tres pasos esenciales por cada microservicio:

1. Agregar la Dependencia: En el archivo **pom.xml** se añade la librería oficial de Springdoc OpenAPI:

```
<dependency>

    <groupId>org.springdoc</groupId>

    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>

    <version>2.5.0</version>

</dependency>
```

2. Configurar el Endpoint de Acceso: En el archivo **application.properties**, se define la ruta personalizada para la interfaz web (opcional, pero recomendado para mantener orden):

Properties

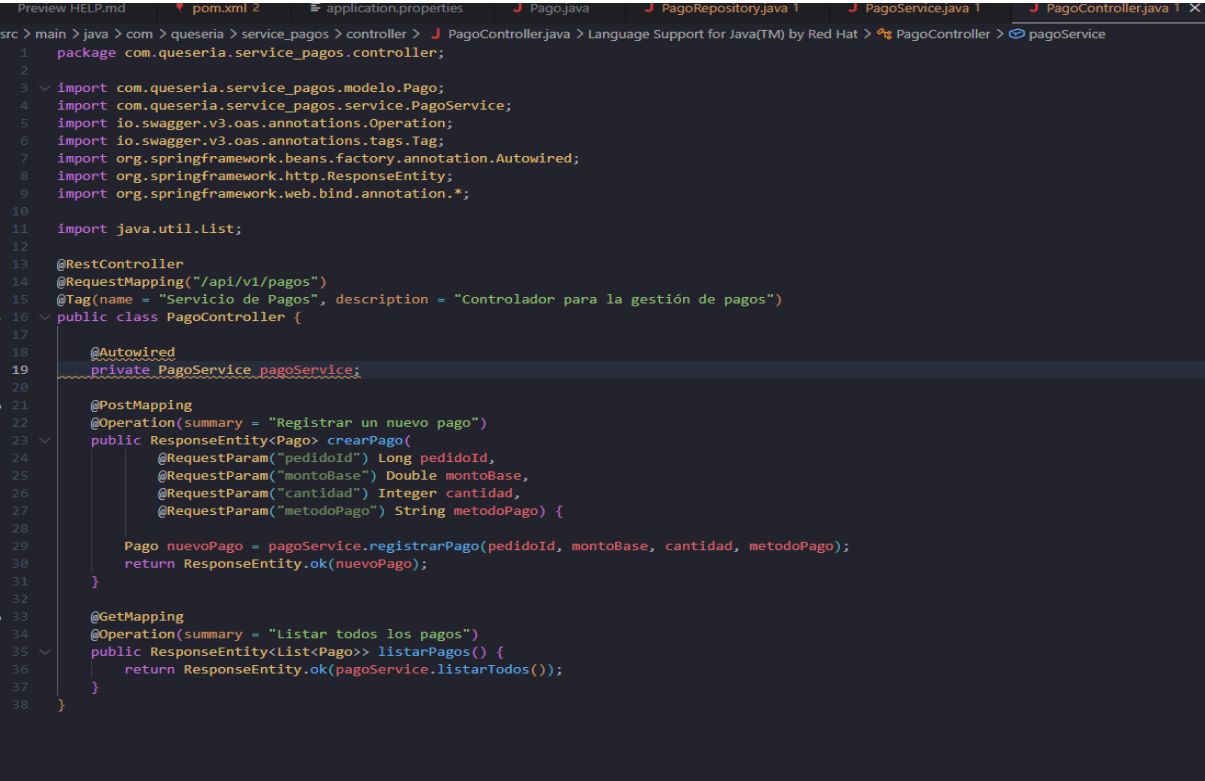
springdoc.api-docs.path=/api-docs

springdoc.swagger-ui.path=/swagger-ui.html

3. Anotar los Controladores: En las clases `@RestController`, se utilizan las anotaciones `@Tag` (para nombrar el módulo) y `@Operation` (para describir qué hace cada método `GET` o `POST`).

Evidencia de Implementación y Ejecución (3 Microservicios)

Microservicio 8: Pagos (`service_pagos`)

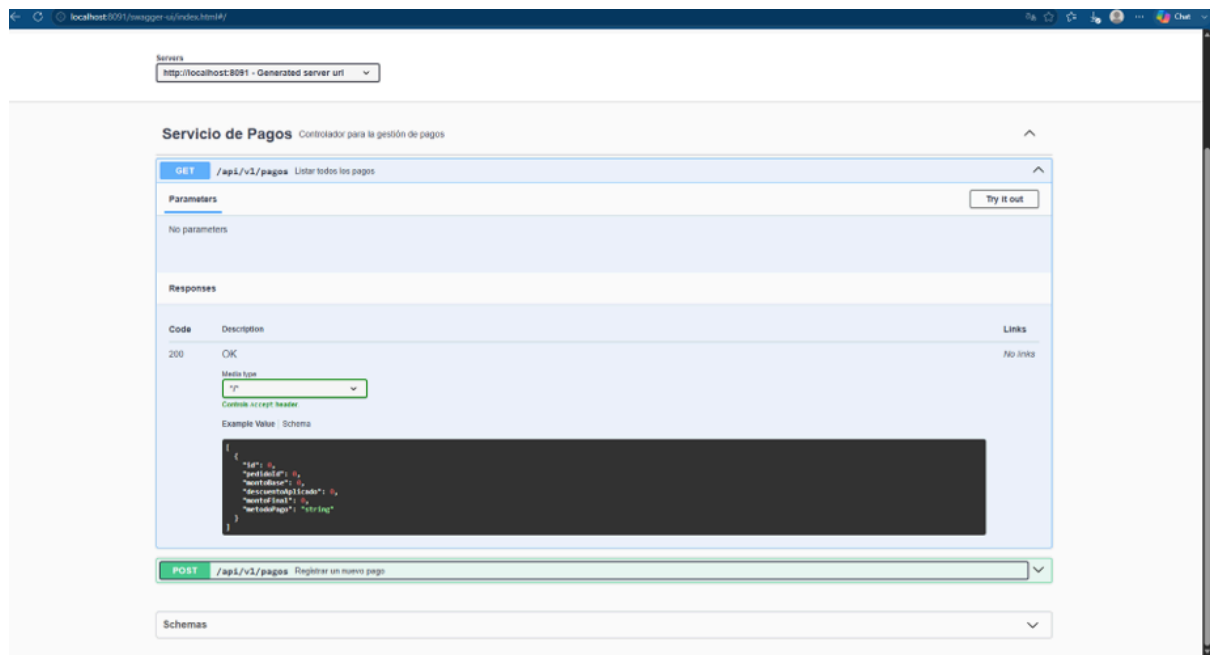


```

1 package com.queseria.service_pagos.controller;
2
3 import com.queseria.service_pagos.modelo.Pago;
4 import com.queseria.service_pagos.service.PagoService;
5 import io.swagger.v3.oas.annotations.Operation;
6 import io.swagger.v3.oas.annotations.tags.Tag;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/api/v1/pagos")
15 @Tag(name = "Servicio de Pagos", description = "Controlador para la gestión de pagos")
16 public class PagoController {
17
18     @Autowired
19     private PagoService pagoService;
20
21     @PostMapping
22     @Operation(summary = "Registrar un nuevo pago")
23     public ResponseEntity<Pago> crearPago(
24         @RequestParam("pedidoId") Long pedidoId,
25         @RequestParam("montoBase") Double montoBase,
26         @RequestParam("cantidad") Integer cantidad,
27         @RequestParam("metodoPago") String metodoPago) {
28
29         Pago nuevoPago = pagoService.registrarPago(pedidoId, montoBase, cantidad, metodoPago);
30         return ResponseEntity.ok(nuevoPago);
31     }
32
33     @GetMapping
34     @Operation(summary = "Listar todos los pagos")
35     public ResponseEntity<List<Pago>> listarPagos() {
36         return ResponseEntity.ok(pagoService.listarTodos());
37     }
38 }

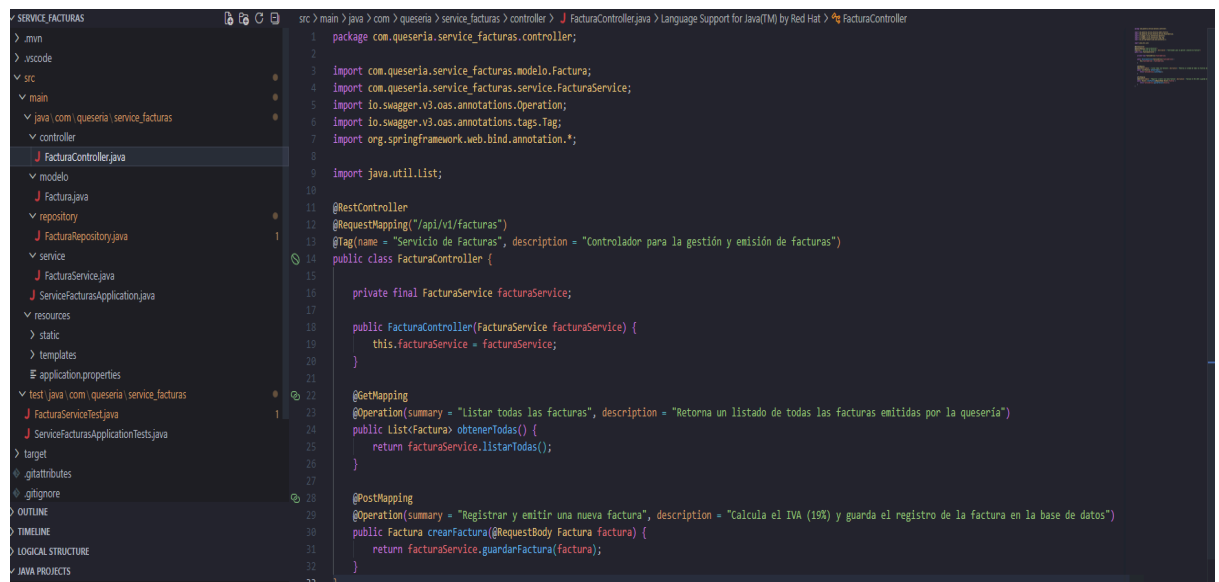
```

- Evidencia en Codificación (Código Fuente):

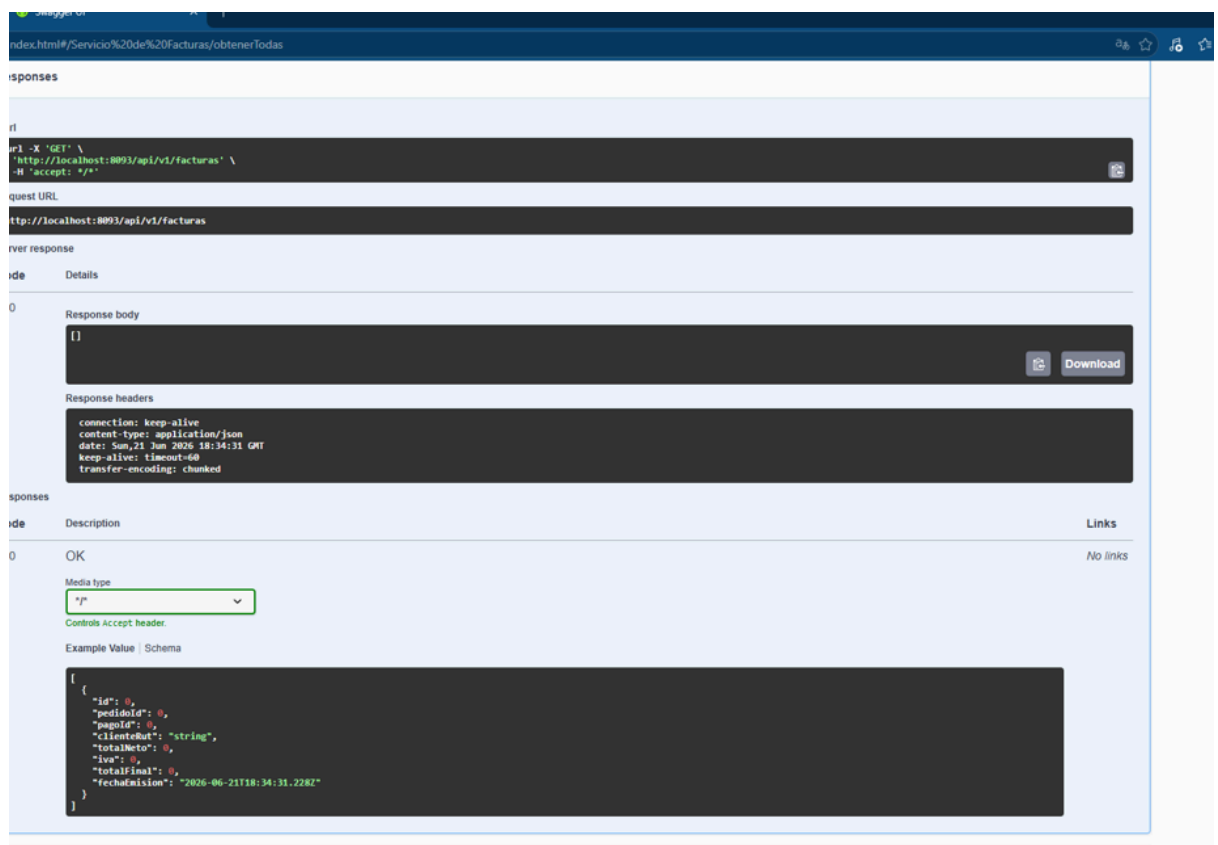
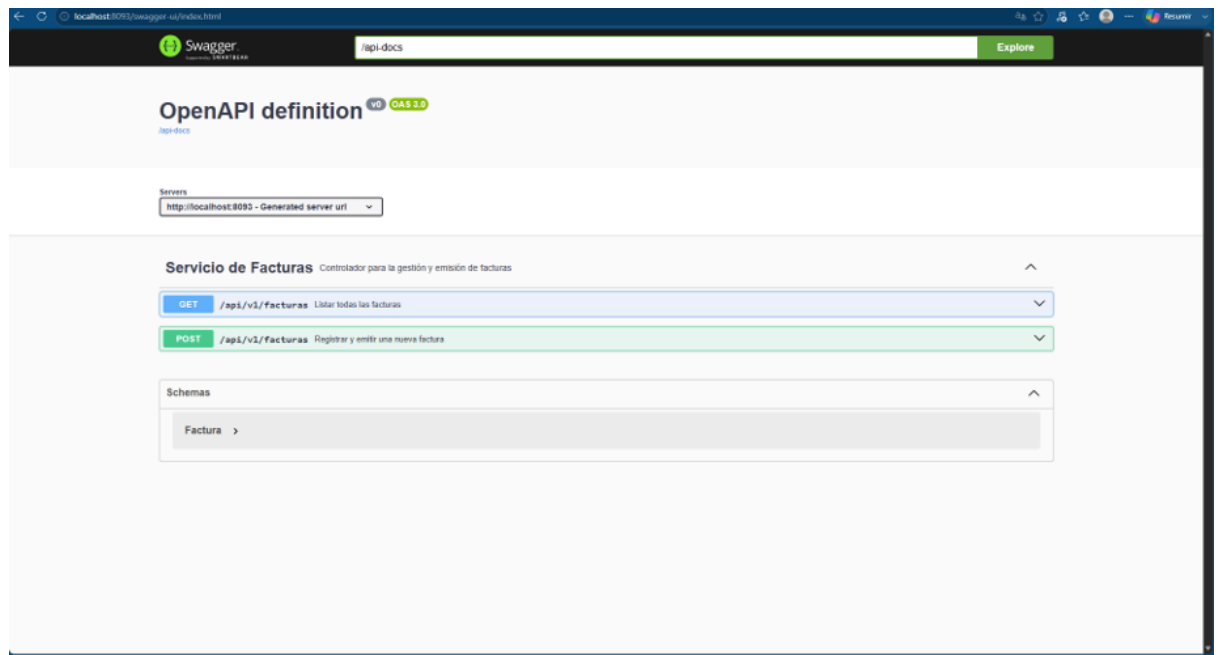


Microservicio 9: Facturas (**service_facturas**)

- Evidencia en Codificación (Código Fuente):

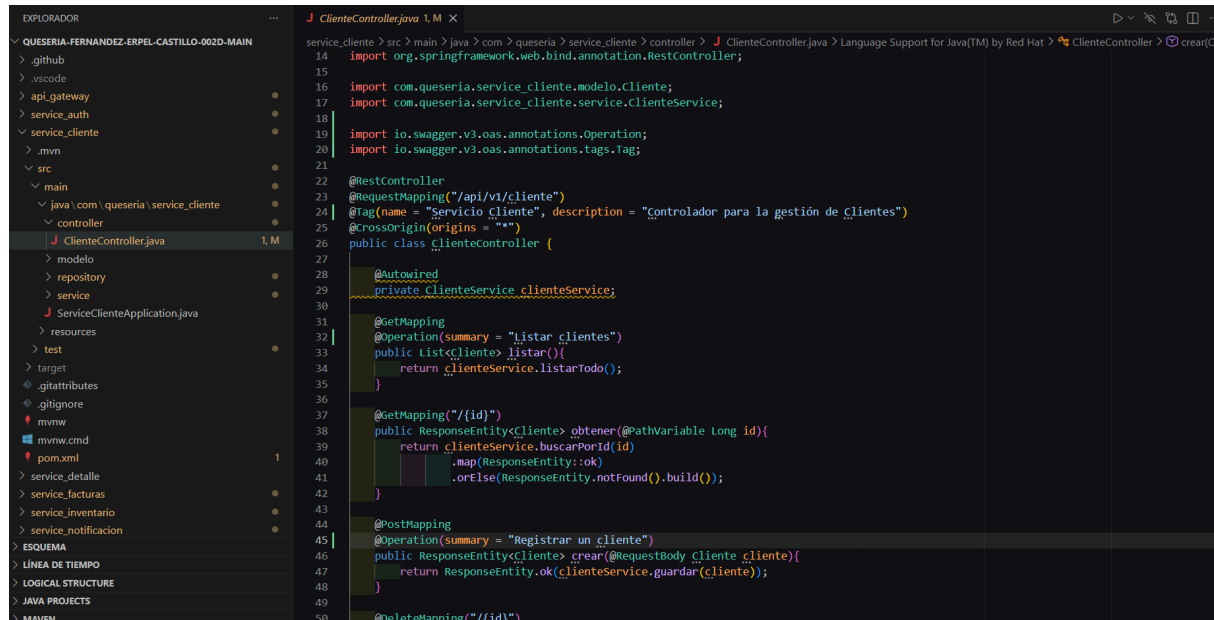


Evidencia en Ejecución (Interfaz Web):



Microservicio 1: Clientes (**service_cliente**)

- Evidencia en Codificación (Código Fuente):

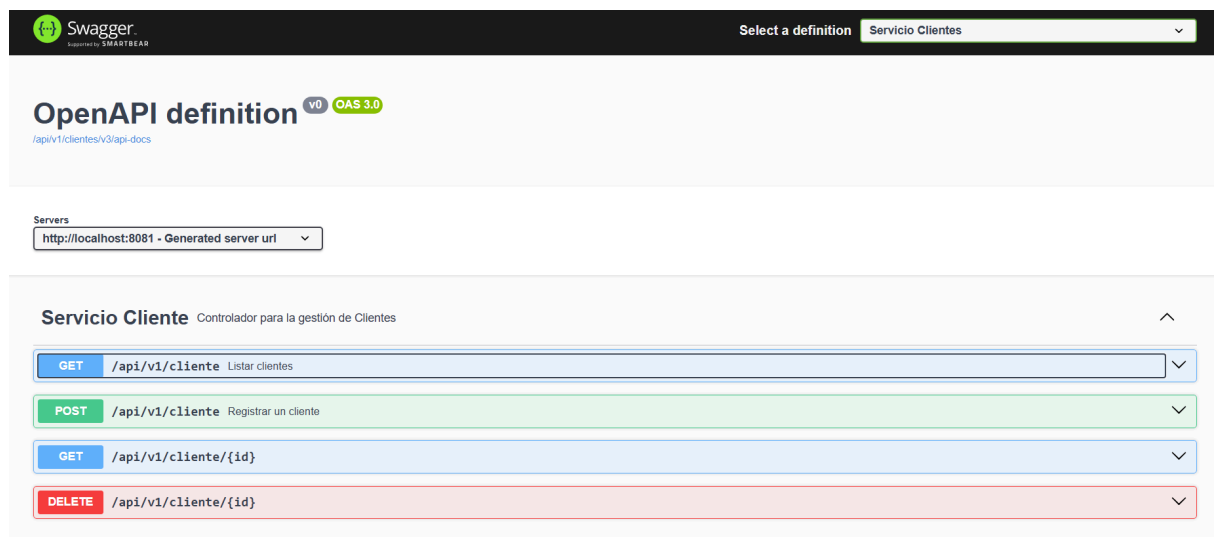


```

14 import org.springframework.web.bind.annotation.RestController;
15
16 import com.queseria.service_cliente.modelo.Cliente;
17 import com.queseria.service_cliente.service.ClientesService;
18
19 import io.swagger.v3.oas.annotations.Operation;
20 import io.swagger.v3.oas.annotations.tags.Tag;
21
22 @RestController
23 @RequestMapping("/api/v1/cliente")
24 @Tag(name = "Servicio cliente", description = "Controlador para la gestión de clientes")
25 @CrossOrigin(origins = "**")
26 public class clienteController {
27
28     @Autowired
29     private ClientesService clienteService;
30
31     @GetMapping
32     @Operation(summary = "Listar clientes")
33     public List<Cliente> listar() {
34         return clienteService.listarTodo();
35     }
36
37     @GetMapping("/{id}")
38     public ResponseEntity<Cliente> obtener(@PathVariable Long id) {
39         return clienteService.buscarPorId(id)
40             .map(ResponseEntity::ok)
41             .orElse(ResponseEntity.notFound().build());
42     }
43
44     @PostMapping
45     @Operation(summary = "Registrar un cliente")
46     public ResponseEntity<Cliente> crear(@RequestBody Cliente cliente) {
47         return ResponseEntity.ok(clienteService.guardar(cliente));
48     }
49
50     @DeleteMapping("/{id}")

```

Evidencia en Ejecución (Interfaz Web):



Swagger

Select a definition: Servicio Clientes

OpenAPI definition v0 OAS 3.0

/api/v1/clientes/v3/api-docs

Servers

http://localhost:8081 - Generated server url

Servicio Cliente Controlador para la gestión de Clientes

GET	/api/v1/cliente	Listar clientes
POST	/api/v1/cliente	Registrar un cliente
GET	/api/v1/cliente/{id}	
DELETE	/api/v1/cliente/{id}	

GET

/api/v1/cliente/{id}

^

Parameters

Cancel

Name	Description
id * required	
integer(\$int64)	2
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/api/v1/cliente/2' \
  -H 'accept: */*'
```

Request URL

http://localhost:8081/api/v1/cliente/2

Server response

Code	Details
200	Response body <pre>{ "id": 2, "nombre": "Marjorie", "email": "peki1234@gmail.com", "telefono": "+5698347363", "direccion": null }</pre> Download